MAINCRAFTS TECHNOLOGY
www.maincrafts.com
hr@maincrafts.com

VLSI Design Internship - Task 5

RTL Design of Datapath, ALU, and Memory Components (RAM/ROM)

Objective

After completing Task-4 (FSM and Control Logic), you now move toward **Datapath Design**, which works together with control logic to form complete digital systems.

The objective of this task is to:

- Understand **datapath architecture**
- Design and implement an **Arithmetic Logic Unit (ALU)**
- Implement **RAM and ROM in Verilog**
- Integrate ALU with basic datapath components
- Learn interaction between **control (FSM) and datapath**
- Simulate and verify system-level RTL design

Why This Task Matters (Industry Context)

- Every processor consists of:
  - **Datapath (computation)**
  - **Control Unit (FSM – Task-4)**
- Key industry components:
  - ALU → arithmetic operations
  - RAM → temporary storage
  - ROM → instruction storage

This task is a **direct step toward CPU design**, which you will build in Task-6 and Task-7.

What You Will Build

You will design and simulate:

1. Arithmetic Logic Unit (ALU)
2. Register File (basic datapath storage)
3. RAM (Read/Write memory)
4. ROM (Read-only memory)
5. Mini Datapath Integration (ALU + Registers)

Each design must include:

- Verilog RTL module
- Testbench
- Simulation waveform

Core Concepts You Will Learn

Concept	Explanation
Datapath	Data processing part of a system
ALU	Performs arithmetic and logic operations
Register File	Collection of registers for fast storage
RAM	Volatile memory (read/write)
ROM	Non-volatile memory (read-only)
Multiplexer	Selects between multiple inputs
Control Signals	Drive datapath operations

Tools & Software (Free)

Use same tools:

- EDA Playground
- Icarus Verilog
- GTKWave
- ModelSim (optional)

Prerequisites

Before starting Task-5:

- Completion of Task-4
- Strong understanding of FSM
- Knowledge of combinational + sequential logic
- Verilog syntax and testbench writing

Part A: ALU Design (Core Component)

Operations to Implement

- Addition
- Subtraction
- AND
- OR
- XOR

Starter Verilog Code

verilog

```verilog
module alu(
    input [3:0] A,
    input [3:0] B,
    input [2:0] sel,
    output reg [3:0] Y
);

always @(*) begin
    case(sel)
        3'b000: Y = A + B;
        3'b001: Y = A - B;
        3'b010: Y = A & B;
        3'b011: Y = A | B;
        3'b100: Y = A ^ B;
        default: Y = 4'b0000;
    endcase
end

endmodule
```

Part B: Register File Design

Objective

Store multiple values and access them using address.

Starter Code

verilog

```verilog
module register_file(
    input clk,
    input we,
    input [1:0] addr,
    input [3:0] data_in,
    output [3:0] data_out
);

reg [3:0] regfile [3:0];

always @(posedge clk) begin
    if (we)
        regfile[addr] <= data_in;
end

assign data_out = regfile[addr];

endmodule
```

Part C: RAM Design (Read/Write Memory)

Starter Code

verilog

```verilog
module ram(
    input clk,
    input we,
    input [1:0] addr,
    input [3:0] data_in,
    output reg [3:0] data_out
);

reg [3:0] memory [3:0];

always @(posedge clk) begin
    if (we)
        memory[addr] <= data_in;
    else
        data_out <= memory[addr];
end

endmodule
```

Part D: ROM Design (Read-Only Memory)

Starter Code

verilog

```verilog
module rom(
    input [1:0] addr,
    output reg [3:0] data_out
);

always @(*) begin
    case(addr)
        2'b00: data_out = 4'b0001;
        2'b01: data_out = 4'b0010;
        2'b10: data_out = 4'b0100;
        2'b11: data_out = 4'b1000;
    endcase
end

endmodule
```

Part E: Mini Datapath Integration

Objective

Connect:

- Register File → ALU → Output

Concept Flow

```
Register A ----\
               → ALU → Output
Register B ----/
```

Starter Code

verilog

```verilog
module datapath(
    input [3:0] A,
    input [3:0] B,
    input [2:0] sel,
    output [3:0] result
);

alu u1(A, B, sel, result);

endmodule
```

Part F: Testbench Example

verilog

```verilog
module tb_alu;

reg [3:0] A, B;
reg [2:0] sel;
wire [3:0] Y;

alu uut(A, B, sel, Y);

initial begin
    A = 4'b0101; B = 4'b0011;

    sel = 3'b000; #10;
    sel = 3'b001; #10;
    sel = 3'b010; #10;
    sel = 3'b011; #10;
    sel = 3'b100; #10;

    $finish;
end

endmodule
```

Part G: Simulation Steps

1. Compile
   `iverilog alu.v tb_alu.v`
2. Run
   `vvp a.out`
3. View waveform
   `gtkwave dump.vcd`
4. Verify:
   - Correct ALU outputs
   - Memory read/write
   - Register updates

Expected Output

After completing Task-5, you should have:

- Working ALU design
- Functional RAM and ROM modules
- Register file implementation
- Integrated datapath
- Simulation waveforms

Deliverables

Mandatory:

1. Verilog files:
   - `alu.v`
   - `register_file.v`
   - `ram.v`
   - `rom.v`
   - `datapath.v`
2. Testbench files
3. Simulation screenshots
4. Short PDF including:
   - Design explanation
   - Results
   - Observations

Optional (Advanced)

- 8-bit or 16-bit ALU
- Pipeline datapath (basic)
- Memory initialization from file
- FSM + datapath integration

Free Learning Resources & Support Guide
(ALU, Register File, RAM, ROM, Datapath Design using Verilog HDL)

How to Use This Guide

To successfully complete Task-5, interns should learn using:

- YouTube tutorials
- Google search (topic-based learning)
- Hands-on coding practice
- Simulation tools
- AI tools for debugging and concept clarity

Instead of relying on limited fixed content, use the **search topics provided below** to explore updated and deeper explanations.

---

1. Datapath Fundamentals (Start Here)

Before implementation, understand how computation happens inside digital systems.

Search topics:

- Datapath design in computer architecture
- Datapath vs control unit
- Basic processor datapath diagram
- Components of datapath (ALU, registers, mux)

Recommended YouTube Learning

Datapath and Control Unit Explained

Introduction to Datapath in Computer Architecture

These videos help you understand:

- How data flows inside a processor
- Role of ALU and registers
- Interaction with control unit

---

2. ALU Design in Verilog

ALU is the core computation unit.

Search topics:

- ALU design using Verilog HDL
- Arithmetic logic unit operations Verilog
- 4-bit ALU Verilog example
- ALU control signals

Recommended YouTube Learning

ALU Design in Verilog (Step-by-Step)

4-bit ALU Verilog Implementation with Explanation

These videos cover:

- Addition, subtraction, logic operations
- Case-based design
- Simulation approach

3. Register File Design

Registers are used for fast temporary storage.

Search topics:

- Register file design in Verilog
- Multi-register system Verilog
- Read write register file example

Recommended YouTube Learning

Register File Design using Verilog HDL

---

4. RAM Design (Read/Write Memory)

Search topics:

- RAM design using Verilog
- Synchronous RAM vs asynchronous RAM
- Memory modeling in Verilog

Recommended YouTube Learning

RAM Design in Verilog (Read/Write Memory)

Verilog RAM Tutorial with Simulation

These videos explain:

- Write enable signal
- Addressing
- Memory storage

---

5. ROM Design (Read Only Memory)

Search topics:

- ROM design in Verilog
- Case-based ROM implementation
- Lookup table design Verilog

Recommended YouTube Learning

ROM Design using Verilog HDL

---

6. Datapath Integration (ALU + Registers)

Now combine all components.

Search topics:

- Datapath design using Verilog
- ALU + register integration
- Basic CPU datapath design
- Multiplexer in datapath

Recommended YouTube Learning

Datapath Design with ALU and Registers

Simple CPU Datapath Explained

---

7. Multiplexer (Important for Datapath Control)

Search topics:

- Multiplexer design Verilog
- MUX in datapath design
- Data selection circuits

Recommended YouTube Learning

Multiplexer (MUX) Design in Verilog

8. Testbench Development

Verification is critical in industry.

Search topics:

- Verilog testbench for ALU
- Testbench for RAM and register file
- Writing stimulus in Verilog

Recommended YouTube Learning

Verilog Testbench Tutorial (Step-by-Step)

---

9. Simulation & Waveform Analysis

Search topics:

- Verilog simulation using Icarus Verilog
- GTKWave waveform analysis
- Debugging digital circuits

Recommended YouTube Learning

GTKWave Tutorial for Waveform Analysis

---

10. Advanced Concepts (Optional but Recommended)

Search topics:

- Pipelined datapath
- ALU flags (zero, carry, overflow)
- Memory initialization in Verilog
- Control signals in datapath

---

11. Free Academic Courses

Search topics:

- NPTEL Computer Architecture
- NPTEL Digital System Design
- VLSI datapath design lectures

---

12. Practice & Mini Projects

To strengthen your understanding, build:

- 8-bit ALU
- Dual-port RAM
- Register-based calculator
- Mini datapath system
- Instruction-based ALU

---

13. Community Support

Search topics:

- Stack Overflow Verilog ALU questions
- Reddit VLSI discussions
- Verilog debugging help